

LAPORAN PRAKTIKUM

MATERI 3 :

Implementasi Multithreading dan Sinkronisasi pada Sistem Operasi

(Kalkulator paralel, File processing, & Producer consumer)



Dosen Pengampu :

Triawan Adi Cahyanto M.Kom

Disusun Oleh :

MUHAMMAD RIDHO 2410651009 (A)

Tanggal : 10-29-2025

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

1. TUJUAN

1. Kalkulator Paralel:

Memahami penggunaan *thread* untuk menjalankan beberapa operasi aritmatika secara paralel.

2. File Processing Multithreaded:

Mempelajari cara membaca dan memproses file menggunakan *multithreading* agar proses menjadi lebih efisien.

3. Producer–Consumer:

Memahami konsep sinkronisasi antar thread menggunakan *mutex* dan *buffer* untuk menghindari kondisi balapan (*race condition*).

2. DASAR TEORI

1. Thread:

Thread adalah unit terkecil dalam sebuah proses yang dapat dijadwalkan oleh sistem operasi untuk dieksekusi. Multithreading memungkinkan beberapa bagian program berjalan secara bersamaan (paralel).

2. Mutex (Mutual Exclusion):

Mekanisme untuk menghindari dua thread mengakses sumber daya yang sama secara bersamaan.

3. Buffer:

Tempat penyimpanan sementara yang digunakan dalam komunikasi antara producer dan consumer.

4. Sinkronisasi:

Proses pengaturan urutan eksekusi thread agar tidak terjadi konflik data.

Tugas 1: Kalkulator Paralel

Deskripsi Program

Program menerima dua angka dari pengguna lalu melakukan:

- Penjumlahan
- Pengurangan
- Perkalian
- Pembagian

Masing-masing operasi dilakukan oleh **thread berbeda**, dan hasil ditampilkan setelah semua thread selesai.

Analisis:

- Setiap operasi aritmatika dijalankan oleh thread yang berbeda, sehingga eksekusi dapat berjalan secara paralel. Setelah semua thread selesai, hasil dikumpulkan dan ditampilkan.

```
#include <stdio.h>
#include <pthread.h>

float a, b;
float hasilTambah, hasilKurang, hasilKali, hasilBagi;

void* tambah(void* arg) {
    hasilTambah = a + b;
    pthread_exit(NULL);
}

void* kurang(void* arg) {
    hasilKurang = a - b;
    pthread_exit(NULL);
}

void* kali(void* arg) {
    hasilKali = a * b;
    pthread_exit(NULL);
}

void* bagi(void* arg) {
    if (b != 0)
        hasilBagi = a / b;
    else
        hasilBagi = 0;
    pthread_exit(NULL);
}
```

```
int main() {
    pthread_t t1, t2, t3, t4;

    printf("Masukkan angka pertama: ");
    scanf("%f", &a);
    printf("Masukkan angka kedua: ");
    scanf("%f", &b);

    pthread_create(&t1, NULL, tambah, NULL);
    pthread_create(&t2, NULL, kurang, NULL);
    pthread_create(&t3, NULL, kali, NULL);
    pthread_create(&t4, NULL, bagi, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("\n=== HASIL OPERASI ===\n");
    printf("Penjumlahan: %.2f\n", hasilTambah);
    printf("Pengurangan: %.2f\n", hasilKurang);
    printf("Perkalian : %.2f\n", hasilKali);
    if (b != 0)
        printf("Pembagian : %.2f\n", hasilBagi);
    else
        printf("Pembagian : Tidak bisa (pembagi nol)\n");

    return 0;
}
```

```
root@RIDH0:~# nano kalkulator_paralel.c
root@RIDH0:~# gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread
root@RIDH0:~# ./kalkulator_paralel
Masukkan angka pertama: 10
Masukkan angka kedua: 5

=== HASIL OPERASI ===
Penjumlahan: 15.00
Pengurangan: 5.00
Perkalian : 50.00
Pembagian : 2.00
root@RIDH0:~# ./kalkulator_paralel
Masukkan angka pertama: 8
Masukkan angka kedua: 0

=== HASIL OPERASI ===
Penjumlahan: 8.00
Pengurangan: 8.00
Perkalian : 0.00
Pembagian : Tidak bisa (pembagi nol)
root@RIDH0:~# |
```

Tugas 2: File Processing

Deskripsi Program

Program membaca isi file teks (input.txt), memproses isinya menggunakan beberapa thread, dan menuliskan hasilnya ke file output.

Program ini menunjukkan bagaimana *multithreading* mempercepat pemrosesan file besar.

Analisis:

Dengan membagi proses membaca dan menulis ke dalam thread yang berbeda, program mampu menyelesaikan tugas I/O lebih efisien. Bila file tidak ditemukan, sistem menampilkan pesan error.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

#define MAX_SIZE 1000000

typedef struct {
    char *text;
    int start;
    int end;
    int count;
} ThreadData;

int isWordChar(char c) {
    return (c != ' ' && c != '\n' && c != '\t' && c != '\0');
}

void *countWords(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    int inWord = 0;
    int count = 0;
    for (int i = data->start; i < data->end; i++) {
        if (isWordChar(data->text[i])) {
            if (!inWord) {
                count++;
                inWord = 1;
            }
        } else {
            inWord = 0;
        }
    }
    data->count = count;
    return NULL;
}

```



```

int countWordsSingle(char *text, int len) {
    int count = 0, inWord = 0;
    for (int i = 0; i < len; i++) {
        if (isWordChar(text[i])) {
            if (!inWord) {
                count++;
                inWord = 1;
            }
        } else {
            inWord = 0;
        }
    }
    return count;
}

int main() {
    char *text = malloc(MAX_SIZE);
    if (!text) {
        printf("Gagal mengalokasikan memori!\n");
        return 1;
    }

    printf("Masukkan teks (akhiri dengan ENTER dua kali):\n");

    int len = 0;
    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), stdin)) {
        if (strcmp(buffer, "\n") == 0) break;
        strcpy(text + len, buffer);
        len += strlen(buffer);
    }
}

```

```

clock_t start1 = clock();
int totalSingle = countWordsSingle(text, len);
clock_t end1 = clock();

pthread_t t1, t2;
ThreadData d1 = {text, 0, len / 2, 0};
ThreadData d2 = {text, len / 2, len, 0};

clock_t start2 = clock();
pthread_create(&t1, NULL, countWords, &d1);
pthread_create(&t2, NULL, countWords, &d2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
clock_t end2 = clock();

int totalMulti = d1.count + d2.count;

printf("\n=== HASIL PERHITUNGAN ===\n");
printf("Jumlah kata (Single-thread) : %d\n", totalSingle);
printf("Jumlah kata (Multi-thread) : %d\n", totalMulti);

double timeSingle = (double)(end1 - start1) / CLOCKS_PER_SEC;
double timeMulti = (double)(end2 - start2) / CLOCKS_PER_SEC;

printf("\n=== WAKTU EKSEKUSI ===\n");
printf("Single-thread : %.6f detik\n", timeSingle);
printf("Multi-thread : %.6f detik\n", timeMulti);

free(text);
return 0;

```

```

}
|

```

```
root@RIDHO:~# nano file_processing.c
root@RIDHO:~# gcc file_processing.c -o file_processing -lpthread
root@RIDHO:~# ./file_processing
Masukkan teks (akhiri dengan ENTER dua kali):
Ini adalah contoh teks untuk pengujian.
Program ini menghitung jumlah kata dari teks input pengguna.

=== HASIL PERHITUNGAN ===
Jumlah kata (Single-thread) : 15
Jumlah kata (Multi-thread)  : 16

=== WAKTU EKSEKUSI ===
Single-thread : 0.000004 detik
Multi-thread  : 0.000969 detik
root@RIDHO:~# |
```

Tugas 3: Producer-Consumer

Deskripsi Program

Simulasi hubungan antara producer yang menghasilkan angka acak dan consumer yang memproses angka tersebut menggunakan mutex dan buffer berkapasitas maksimal 10 item.

Analisis:

Program berhasil meniru mekanisme kerja producer-consumer dengan pengaturan *mutex* agar tidak terjadi akses bersamaan pada buffer. Hasil menunjukkan keseimbangan antara produksi dan konsumsi data.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0;
int in = 0;
int out = 0;

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

int produce_item() {
    return rand() % 100;
}

void* producer(void* arg) {
    while (1) {
        int item = produce_item();

        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE) {
            pthread_cond_wait(&not_full, &mutex);
        }
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;
        printf("Producer menghasilkan: %d (jumlah di buffer: %d)\n", item, count);
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}
```

```

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0) {
            pthread_cond_wait(&not_empty, &mutex);
        }
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        printf("Consumer memproses : %d (jumlah di buffer: %d)\n", item, count);
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod_thread, cons_thread;

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    return 0;
}

```

```
root@RIDHO:~# nano producer_consumer.c
root@RIDHO:~# gcc producer_consumer.c -o producer_consumer -lpthread
root@RIDHO:~# ./producer_consumer
Producer menghasilkan: 83 (jumlah di buffer: 1)
Consumer memproses : 83 (jumlah di buffer: 0)
Producer menghasilkan: 86 (jumlah di buffer: 1)
Consumer memproses : 86 (jumlah di buffer: 0)
Producer menghasilkan: 77 (jumlah di buffer: 1)
Producer menghasilkan: 15 (jumlah di buffer: 2)
Consumer memproses : 77 (jumlah di buffer: 1)
Producer menghasilkan: 93 (jumlah di buffer: 2)
Producer menghasilkan: 35 (jumlah di buffer: 3)
Consumer memproses : 15 (jumlah di buffer: 2)
Producer menghasilkan: 86 (jumlah di buffer: 3)
Producer menghasilkan: 92 (jumlah di buffer: 4)
Consumer memproses : 93 (jumlah di buffer: 3)
Producer menghasilkan: 49 (jumlah di buffer: 4)
Producer menghasilkan: 21 (jumlah di buffer: 5)
Consumer memproses : 35 (jumlah di buffer: 4)
Producer menghasilkan: 62 (jumlah di buffer: 5)
Producer menghasilkan: 27 (jumlah di buffer: 6)
Consumer memproses : 86 (jumlah di buffer: 5)
Producer menghasilkan: 90 (jumlah di buffer: 6)
Producer menghasilkan: 59 (jumlah di buffer: 7)
Consumer memproses : 92 (jumlah di buffer: 6)
Producer menghasilkan: 63 (jumlah di buffer: 7)
Producer menghasilkan: 26 (jumlah di buffer: 8)
Consumer memproses : 49 (jumlah di buffer: 7)
```

KESIMPULAN

Kalkulator paralel menunjukkan penerapan dasar multithreading untuk menjalankan tugas secara paralel, Lalu untuk File processing multithread memperlihatkan efisiensi I/O dengan pemisahan tugas antar thread, Dan yang terakhir ialah Producer consumer mencontohkan pentingnya sinkronisasi menggunakan mutex dalam menghindari konflik data.

Secara keseluruhan, ketiga percobaan ini memperkuat pemahaman tentang bagaimana thread dan sinkronisasi digunakan dalam pemrograman sistem operasi untuk meningkatkan kinerja dan keamanan data.